

EC2 Complete Notes

Amazon Elastic Compute Cloud

Owner: Om Chaudhari

© All Rights Reserved

Not for distribution or unauthorized copying

Generated: August 26, 2026

page 1

Table of Contents

1. What is EC2?
2. Why EC2?
3. EC2 Use Cases
4. EC2 Instance
5. EC2 Instance Types
6. General Purpose Instances
7. Burstable Instances (T Family)
8. Compute Optimized (C Family)
9. Memory Optimized (R/X/U/Z)
10. Storage Optimized (I/D)
11. Accelerated Computing
12. HPC Optimized
13. Instance Family Naming
14. Processor Options
15. Instance Generation
16. Instance Size
17. New Instance Families
18. AMI (Amazon Machine Image)
19. EC2 Instance States
20. Start, Stop, Reboot, Terminate
21. EC2 Purchasing Options
22. AWS Nitro System
23. CPU Credits
24. EC2 Instance Lifecycle
25. Monitoring EC2 with CloudWatch
26. EC2 Scaling
27. Auto Scaling Groups
28. EC2 and High Availability

29. EC2 User Data

30. EC2 Instance Metadata (IMDS)

31. EC2 Instance Profiles & IAM Roles

32. Region, Availability Zone & Placement

1. What is EC2?

Amazon EC2 (Elastic Compute Cloud) provides resizable compute capacity in AWS. EC2 lets you create and manage virtual servers in the AWS Cloud. An EC2 virtual server is called an **EC2 instance**.

When configuring EC2, you can choose: CPU, Memory, Instance size, Processor architecture, Operating system, Instance family, and Pricing/purchasing option. The main benefit is that you don't need to purchase physical servers yourself.

2. Why EC2?

- On-demand compute capacity
- Flexible instance configurations
- Ability to scale compute resources
- Multiple processor choices
- Different pricing models
- Ability to start/stop/terminate resources
- Integration with AWS services
- Specialized hardware for AI/ML and HPC

Page 4

3. EC2 Use Cases

Use Case	Examples
Web Applications	Web servers, Application servers, APIs, Backend applications
Development & Testing	Development, Testing, CI/CD, Experiments
Batch Processing	CPU-intensive workloads
High-Performance Computing	Scientific calculations, Simulations, Engineering workloads
Machine Learning/AI	GPU/Trainium/Inferentia instances
Gaming	Gaming servers with high CPU/network performance

4. EC2 Instance

An **EC2 instance** is a virtual server. When launching one, you select an **instance type**, which determines the resources available: vCPUs, Memory, Network performance, EBS bandwidth, and Accelerators.

Example instance types:

- t3.micro
- m7i.large
- c7i.2xlarge

5. EC2 Instance Types

AWS provides different instance families because different workloads need different resources. Here are the main categories:

Category	Designed For
General Purpose	Balanced workloads
Compute Optimized	CPU-intensive workloads
Memory Optimized	Memory-intensive workloads
Storage Optimized	High local-storage/I/O workloads
Accelerated Computing	GPU/accelerator workloads
HPC Optimized	High-performance computing

6. General Purpose Instances

Provide a balance of Compute, Memory, and Networking. Suitable for a wide variety of applications including web servers, application servers, small/medium databases, development, and testing.

Main families: M and T

7. Burstable Instances — T Family

T-family instances are burstable performance instances, designed for workloads where CPU usage is normally low/moderate but occasionally needs to increase. They use **CPU credits** to provide burst performance.

Useful for: Small web servers, Development environments, Small applications, Low/variable CPU workloads

8. Compute Optimized — C Family

Provide a high ratio of compute power to memory. Designed for CPU-intensive workloads such as batch processing, scientific modeling, gaming servers, high-performance web servers, video encoding, and CPU-intensive applications.

9. Memory Optimized — R/X/U/Z

Designed for applications requiring large amounts of RAM. Suitable for in-memory databases, real-time analytics, large caches, big-data processing, and memory-intensive applications.

10. Storage Optimized — I/D Families

Designed for workloads requiring high-performance local storage such as data warehousing, log processing, distributed file systems, and high read/write workloads.

11. Accelerated Computing

Use specialized hardware to accelerate specific workloads. Accelerators include GPUs, FPGAs, AWS Inferentia, and AWS Trainium. Used for machine learning, AI training/inference, graphics, video processing, and scientific workloads.

Important families: G (GPU/graphics), P (GPU), F (FPGA), Inf (Inferentia), Trn (Trainium)

12. HPC Optimized

Hpc instances are designed for high-performance computing. Used for scientific simulations, engineering, computational workloads, research, and large-scale simulations.

13. EC2 Instance Family Naming

AWS follows a consistent naming pattern. Example: **m7i.2xlarge**

Component	Value	Meaning
Family	m	General Purpose
Generation	7	7th Generation
Processor	i	Intel
Size	2xlarge	Instance Size

14. Processor Options

EC2 offers instances based on different processor architectures:

Processor	Represented By	Example
Intel	i	m7i
AMD	a	m7a
AWS Graviton	g	m7g (Arm-based)

15. Instance Generation

The number in the instance name indicates the generation. Example: m6i (6th), m7i (7th), m8i (8th). Newer generations generally bring improvements in performance, price-performance, networking, and CPU capabilities. **Learn the naming pattern, don't memorize every generation.**

16. Instance Size

An instance family is available in multiple sizes (m7i.large, m7i.xlarge, m7i.2xlarge, m7i.4xlarge, etc.). Larger sizes provide more vCPUs, Memory, Network capability, and Compute capacity, allowing you to choose based on your workload.

17. New EC2 Instance Families

AWS continuously adds newer generations and families. Modern EC2 includes M8, C8, R8, I8, newer Graviton generations, newer Trainium/Inferentia generations, and newer GPU-based instances. **Learn the family concepts rather than memorizing today's catalog.**

18. AMI — Amazon Machine Image

An **AMI** is a template used to launch EC2 instances. An AMI can contain: Operating system, Pre-installed software, Application configuration, and Other required configuration. You can use AWS-provided AMIs, AWS Marketplace AMIs, Community AMIs, or custom AMIs.

19. EC2 Instance States

An EC2 instance can move through different states:

Pending: AWS is preparing the instance

Running: The instance is actively running

Stopping: AWS is stopping the instance

Stopped: The instance is powered off (can be restarted)

Shutting-down: The instance is being terminated

Terminated: The instance has been permanently terminated

20. Start, Stop, Reboot & Terminate

Start: Starts a stopped EC2 instance

Stop: Stops the instance (useful when compute capacity not needed temporarily)

Reboot: Restarts the instance

Terminate: Permanently removes the EC2 instance

Remember: Stop = power off, Terminate = delete

21. EC2 Purchasing Options

On-Demand: Pay for compute without commitment. Best for unpredictable workloads and experimentation.

Reserved Instances: Commit to 1-3 year terms for discounted pricing. Useful for predictable workloads.

Savings Plans: ■ Provide discounted compute for 1-3 year commitment. More flexible than Reserved. Two types: Compute Savings Plans (more flexible) and EC2 Instance Savings Plans (specific to family/Region).

Spot Instances: Use spare AWS capacity at significant discount. Tradeoff: AWS can interrupt Spot instances. Suitable for batch jobs, data processing, CI/CD, and fault-tolerant workloads.

Capacity Reservations: Reserve EC2 capacity in specific Availability Zone (capacity commitment, not pricing).

Dedicated Hosts: Physical server dedicated to your use. For licensing and compliance requirements.

Dedicated Instances: EC2 instances on hardware dedicated to single customer.

22. AWS Nitro System

Page 10

The **AWS Nitro System** is the underlying AWS hardware/software architecture used by many modern EC2 instances. It provides high performance, security, efficient virtualization, and high networking performance with reduced virtualization overhead. Includes Nitro Cards, Nitro Security Chip, and Nitro Hypervisor. **Remember: Nitro is the modern platform powering modern EC2.**

23. CPU Credits ■

CPU credits are particularly important for T-family burstable instances. They allow a burstable instance to temporarily use more CPU performance. Instances earn credits during normal usage and use credits to burst when CPU demand increases. This is why T instances are useful for workloads with occasional CPU spikes rather than consistently high CPU workloads.

24. EC2 Instance Lifecycle

A simple EC2 lifecycle includes: Launch → Pending → Running → (optionally Reboot, Stop, or Terminate)

25. Monitoring EC2 with CloudWatch

EC2 can be monitored using **Amazon CloudWatch**. You can monitor CPU utilization, Network activity, Instance status, EBS-related metrics, and System performance. **CloudWatch helps you understand how your EC2 instance is performing.**

26. EC2 Scaling

Vertical Scaling: Increase the size of an existing instance (small → large, more CPU/RAM)

Horizontal Scaling: Add more EC2 instances (1 EC2 → 2 EC2 → 5 EC2). Commonly automated using **Auto Scaling Groups**.

27. Auto Scaling Groups (ASG)

An **Auto Scaling Group** automatically manages a group of EC2 instances. It can launch instances, terminate instances, maintain desired capacity, scale based on demand, and replace unhealthy instances. Example: When traffic increases, ASG launches more instances; when traffic decreases, ASG scales down.

28. EC2 and High Availability

A single EC2 instance can fail. Production applications commonly use multiple EC2 instances for redundancy. Combined with Auto Scaling, EC2 can automatically replace failed instances and adjust capacity.

29. EC2 User Data ■

User Data allows you to provide commands/scripts that run when an instance is launched. For example, you can automatically install a web server. This is extremely useful for automation.

```
#!/bin/bash
dnf install -y nginx
systemctl enable nginx
systemctl start nginx
```

30. EC2 Instance Metadata (IMDS) ■

EC2 provides an **Instance Metadata Service (IMDS)** that allows an instance to access information about itself (instance identity, network information, IAM role credentials). Modern AWS practice prefers **IMDSv2** because it provides stronger protections against credential-access attacks. **IMDS = information about the running EC2 instance.**

31. EC2 Instance Profiles & IAM Roles

EC2 applications often need to access AWS services. Instead of putting AWS access keys directly inside the server, attach an **IAM role to the EC2 instance** through an instance profile. This is the recommended approach for giving applications running on EC2 AWS permissions.

32. Region, Availability Zone & Placement

AWS Region: A separate geographic area where AWS has infrastructure. Examples: ap-south-1 (Mumbai), us-east-1 (N. Virginia). Choose based on: user location, compliance requirements, service availability, pricing, disaster recovery.

Availability Zone (AZ): An isolated location within an AWS Region. Example: ap-south-1a, ap-south-1b. Purpose: Fault isolation. If AZ-A fails, instances in AZ-B/C can continue.

Region vs AZ:

- Region = geographic area (ap-south-1)
- AZ = isolated location inside Region (ap-south-1a)
- Region contains multiple AZs
- AZ belongs to one Region

Not Every Instance Type Exists Everywhere: Each Region supports only a subset of EC2 instance types. Instance availability may vary by AZ within a Region.

Local Zones: AWS infrastructure placed closer to end users than parent Region. Reduce latency for supported workloads.

Wavelength Zones: AWS infrastructure within 5G networks. For 5G applications and ultra-low-latency workloads.

AWS Outposts: Extends AWS infrastructure into customer data centers. For on-premises AWS services.

EC2 Placement Groups: ■ Allow you to influence where EC2 instances are physically placed relative to one another.

Placement Group Types:

- **Cluster:** Instances close together in single AZ. Very low-latency, high-throughput. For HPC, tightly coupled workloads.
- **Partition:** Separate partitions with distinct hardware. Reduce correlated failures. For Hadoop, Cassandra, Kafka.
- **Spread:** Instances on distinct hardware. Reduce single hardware failure impact. For small critical instances.
- **Precision Time:** Hardware with high-precision time sources. For time-critical applications.

Key Concepts Summary

Core EC2 Concepts:

- ✓ EC2 definition and benefits
- ✓ EC2 instances and instance types
- ✓ Instance families (General, Compute, Memory, Storage, Accelerated, HPC)
- ✓ Instance naming convention
- ✓ Processor options (Intel, AMD, Graviton)
- ✓ Instance generations and sizes

Instance Management:

- ✓ Launch, Start, Stop, Reboot, Terminate operations
- ✓ Instance states and lifecycle
- ✓ AMI (Amazon Machine Image)
- ✓ User Data for automation
- ✓ Instance Metadata (IMDS)
- ✓ IAM Roles and Instance Profiles

Performance & Architecture:

- ✓ vCPU, Memory, and CPU credits
- ✓ AWS Nitro System
- ✓ Vertical and horizontal scaling
- ✓ Auto Scaling Groups
- ✓ High availability patterns

Purchasing & Deployment:

- ✓ On-Demand, Reserved, Savings Plans, Spot
- ✓ Capacity Reservations and Dedicated options
- ✓ Region and Availability Zone selection
- ✓ Placement Groups
- ✓ Local Zones, Wavelength, Outposts

Monitoring:

- ✓ CloudWatch metrics and monitoring

Topics NOT Covered in EC2-Specific Notes:

- ✗ VPC, Subnets, Route Tables
- ✗ EBS (covered separately)
- ✗ Elastic Load Balancer
- ✗ Route 53
- ✗ CloudWatch in depth
- ✗ Security Groups in depth